

Short-time quantum Fourier transform processing

ICASSP 2025 (Hyderabad, India)

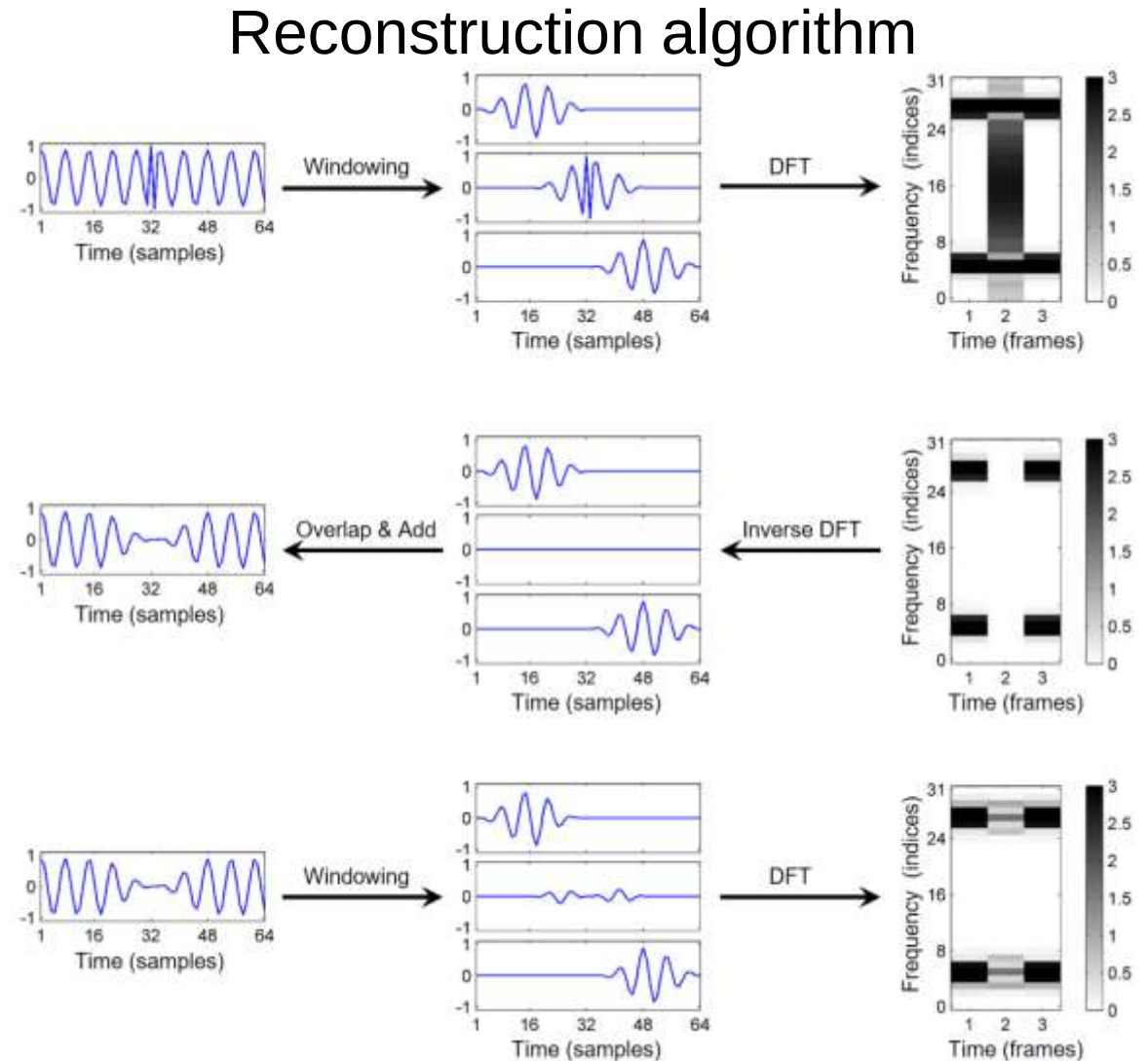
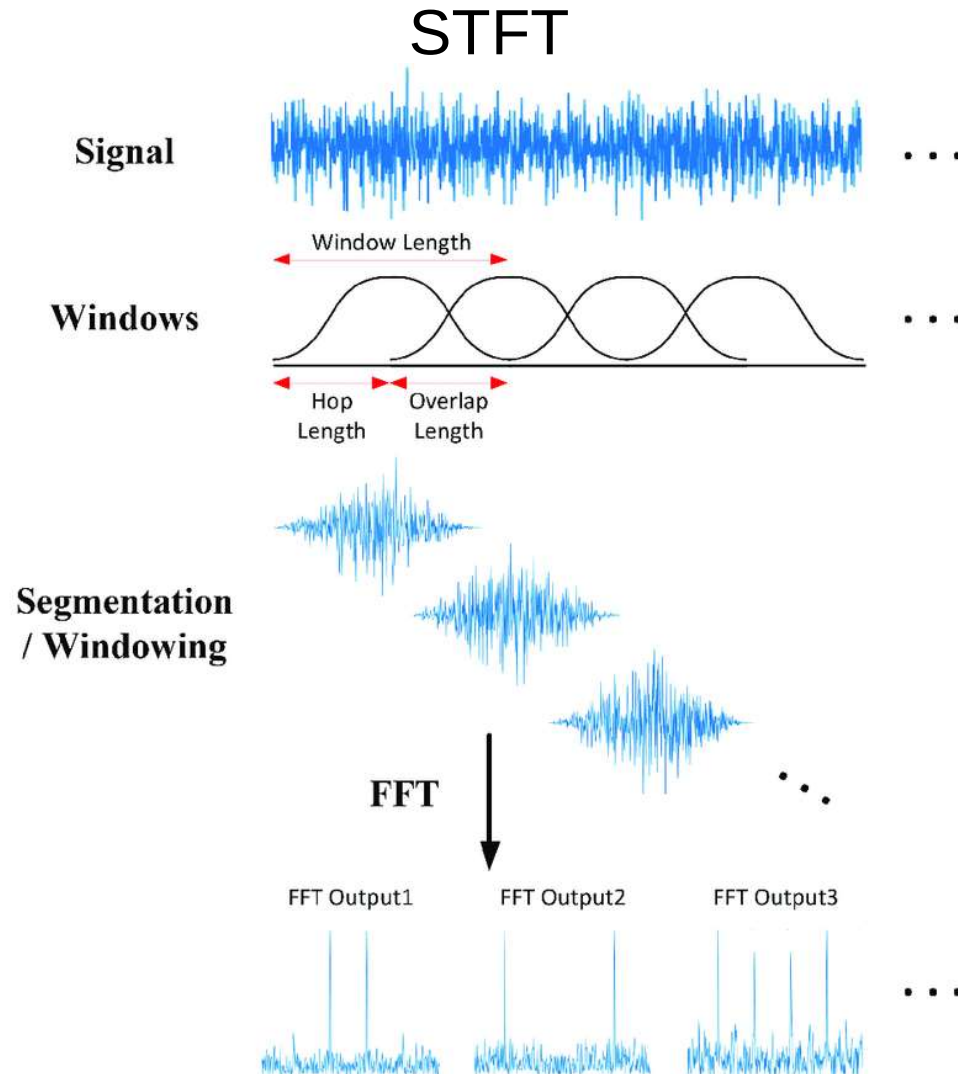


PRESENTER : SREERAJ RAJINDRAN NAIR (UTS, DOLBY INTERN)

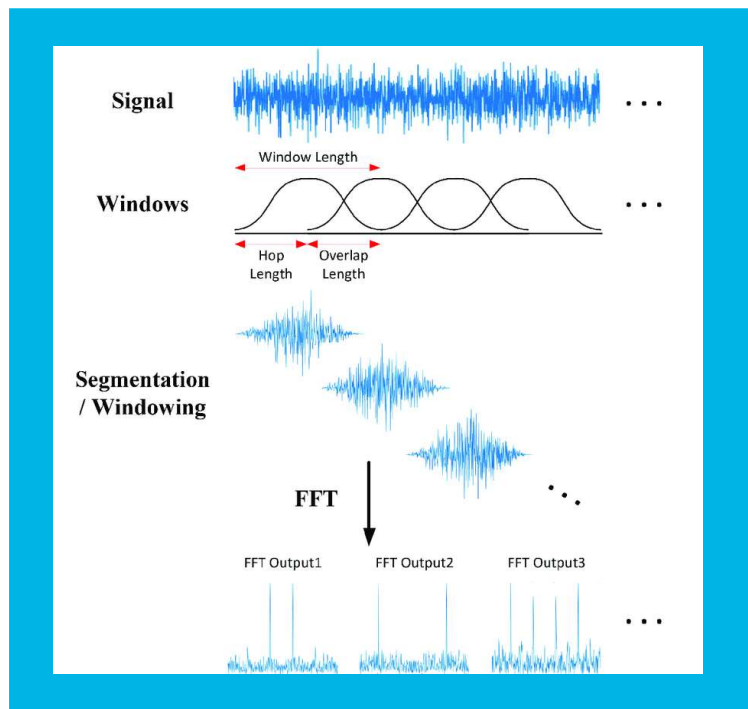
CO-AUTHORS : BENJAMIN SOUTHWELL (DOLBY), CHRISTOPHER FERRIE (UTS)

- Proposal: To employ Quantum windowing in conjunction with quantum adapted version of classical signal reconstruction algorithm and demonstrate quantum speedup/advantage (if any). Benchmark traditional windowing functions in QWFT.
- Motivation: Successful use of Quantum algorithms for signal processing will require this reconstruction algorithm. Rolling codec in conjunction with Quantum algorithms will need this as a subroutine.
- Novelty: Quantum windowing is a relatively less developed aspect of Quantum signal processing. It is crucial to bridge this gap in research. Reconstruction has never been attempted with QWFT.

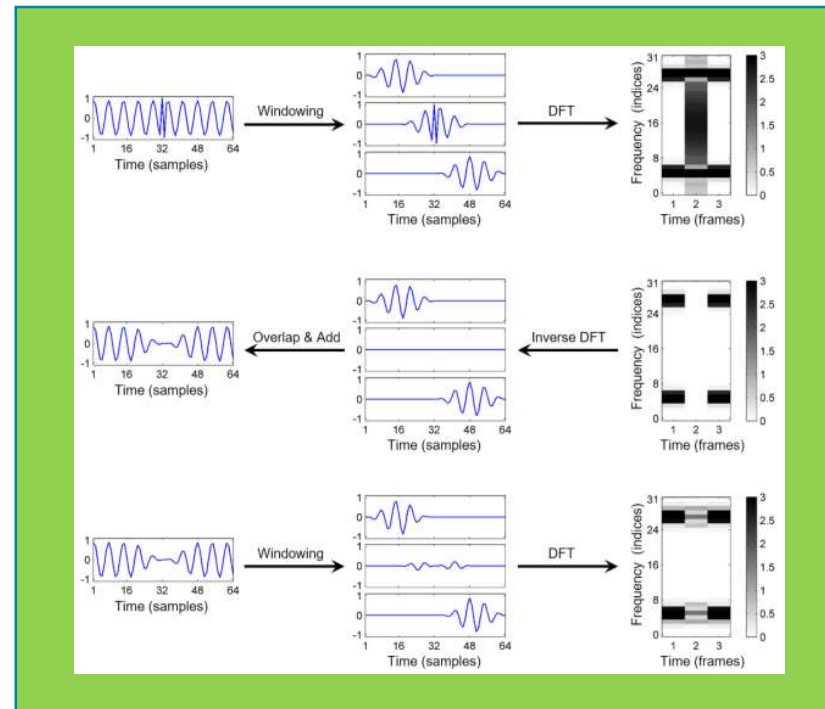
OBJECTIVE: Adapt Classical Reconstruction framework to Quantum domain



QWFT (mod)

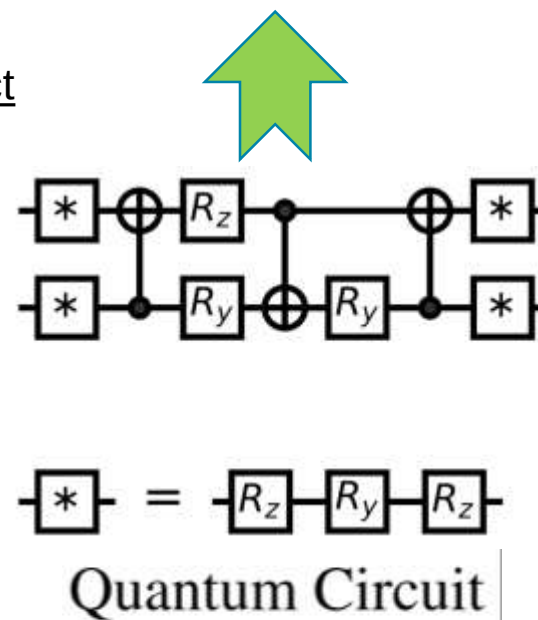


Quantum Reconstruction algorithm



Simulations
+
Real Hardware

Proposed framework for the project



Quantum information

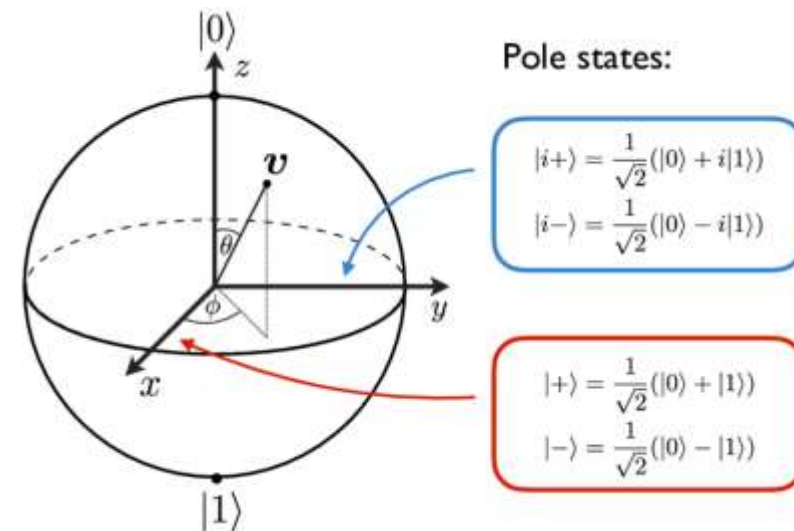
A **quantum state** of a system is represented by a **column vector** whose indices are placed in correspondence with the classical states of that system:

- The entries are complex numbers.
- The sum of the absolute values squared of the entries must equal 1.

Definition

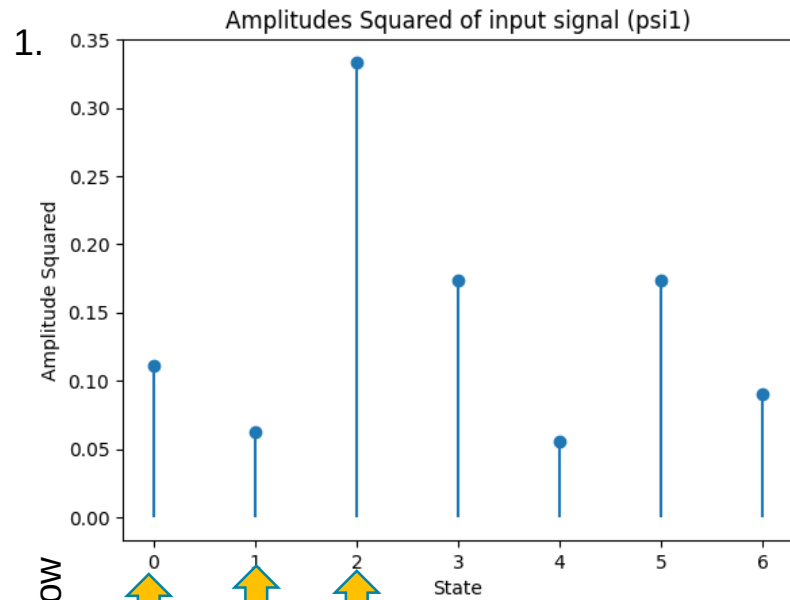
The **Euclidean norm** for vectors with complex number entries is defined like this:

$$\mathbf{v} = \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{pmatrix} \Rightarrow \|\mathbf{v}\| = \sqrt{\sum_{k=1}^n |\alpha_k|^2}$$



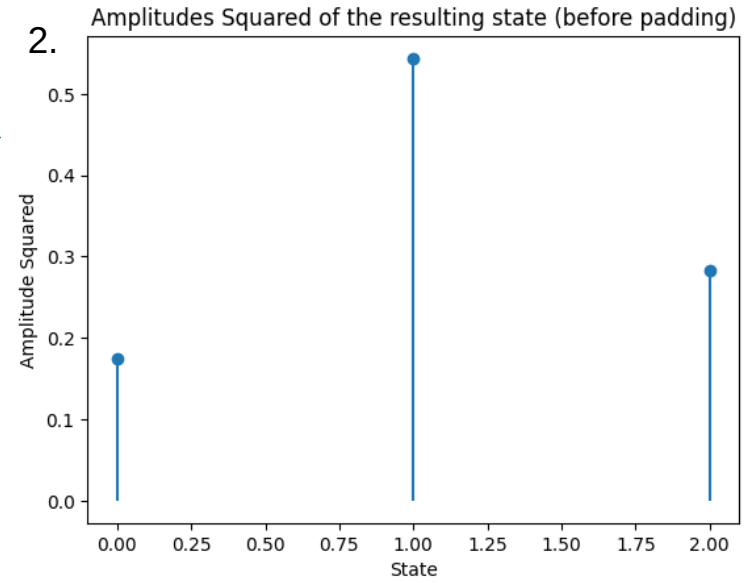
Quantum state vectors are therefore **unit vectors** with respect to this norm.

Quantum Window State preparation:

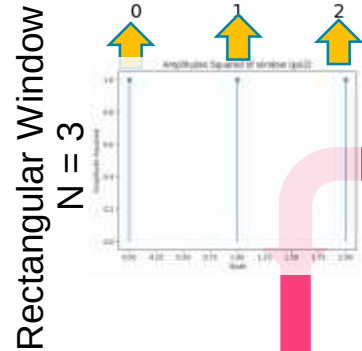


Element wise product and L2 Normalisation
TODO: TRACK WINDOW NORM

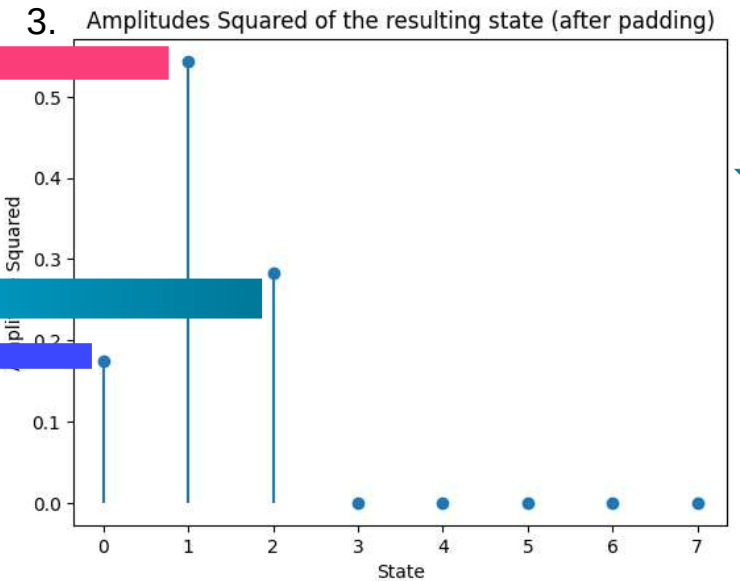
Classical Preprocessing



Padding to the power of 2 over (N + M - 1)



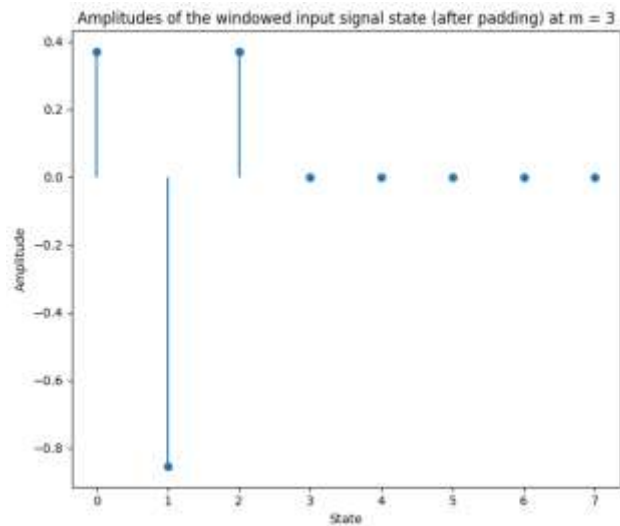
Quantum Processing
Amplitude Encoding



$$|\Psi\rangle = a|000\rangle + b|001\rangle + c|010\rangle + d|011\rangle + e|100\rangle + f|101\rangle + g|110\rangle + h|111\rangle$$

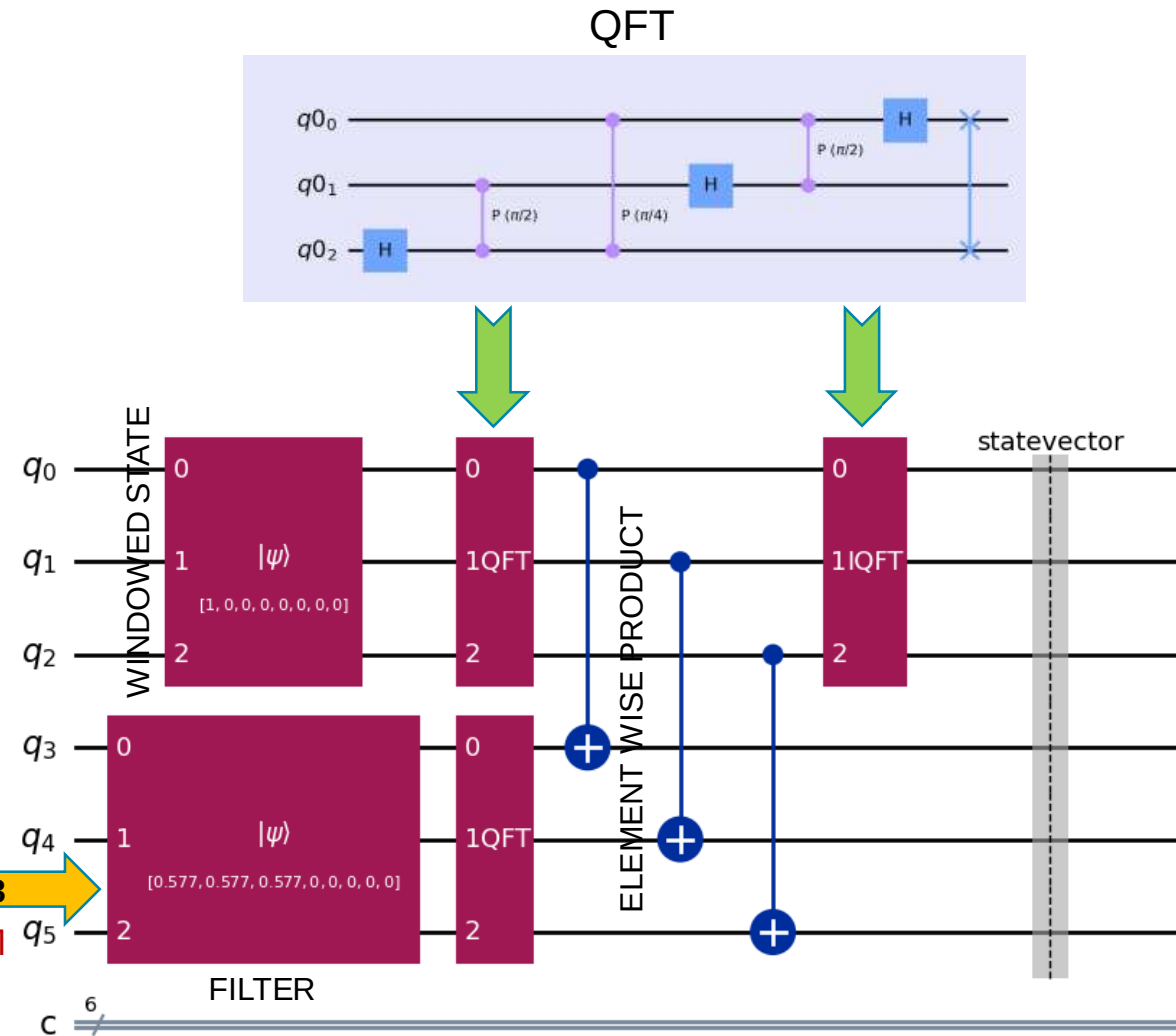
SUBROUTINE 1: FILTER AS A QUANTUM REGISTER

Subroutine 1: Quantum Convolution algorithm: Filter as a Quantum Register



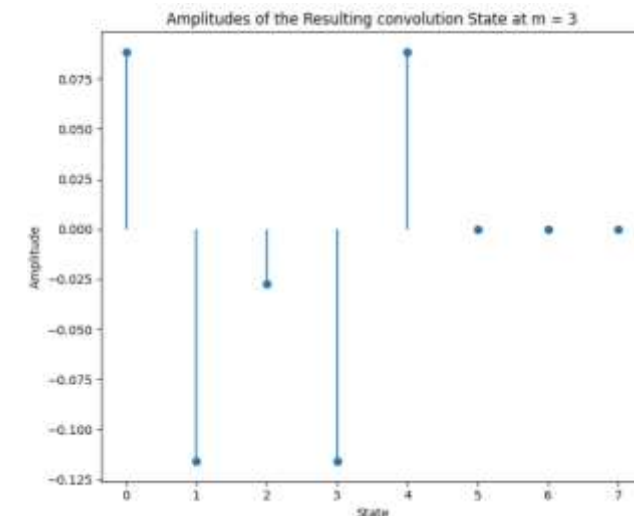
Box car filter / Avg filter. N = 3

TODO: TRACK FILTER NORM



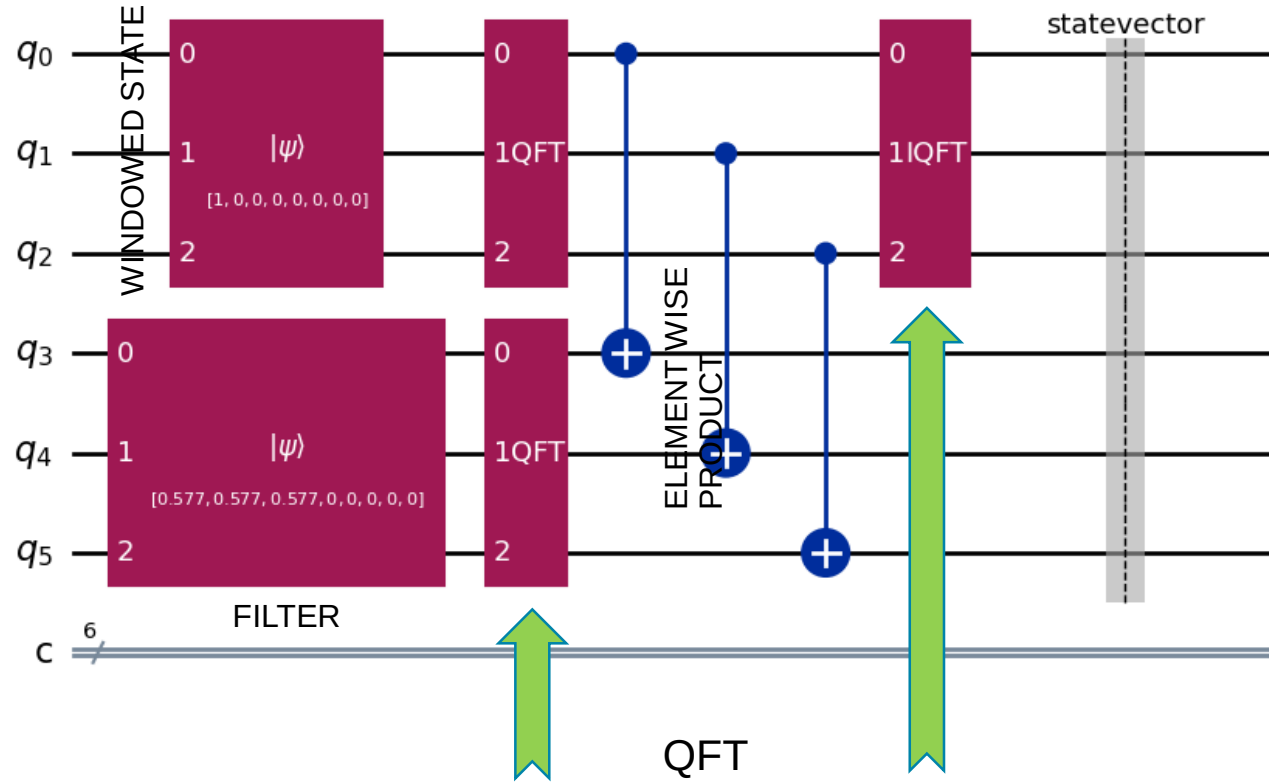
For N qubit window register we need N ancillas to handle filter encoding. So 2N qubits in total.

O/P: CONVOLUTION

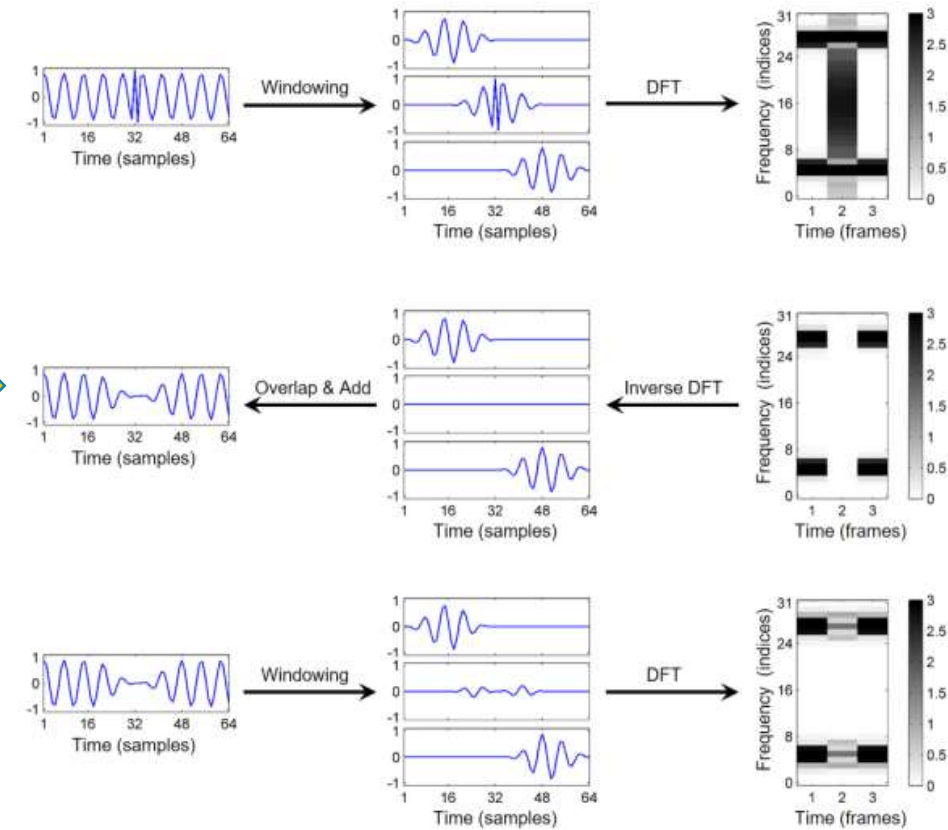


TODO:
RECTIFY WINDOW NORM
and
FILTER NORM

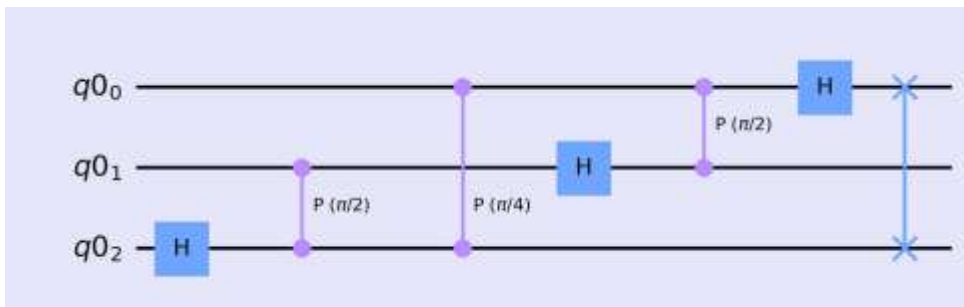
Quantum Reconstruction algorithm: Classical Overlap-Add



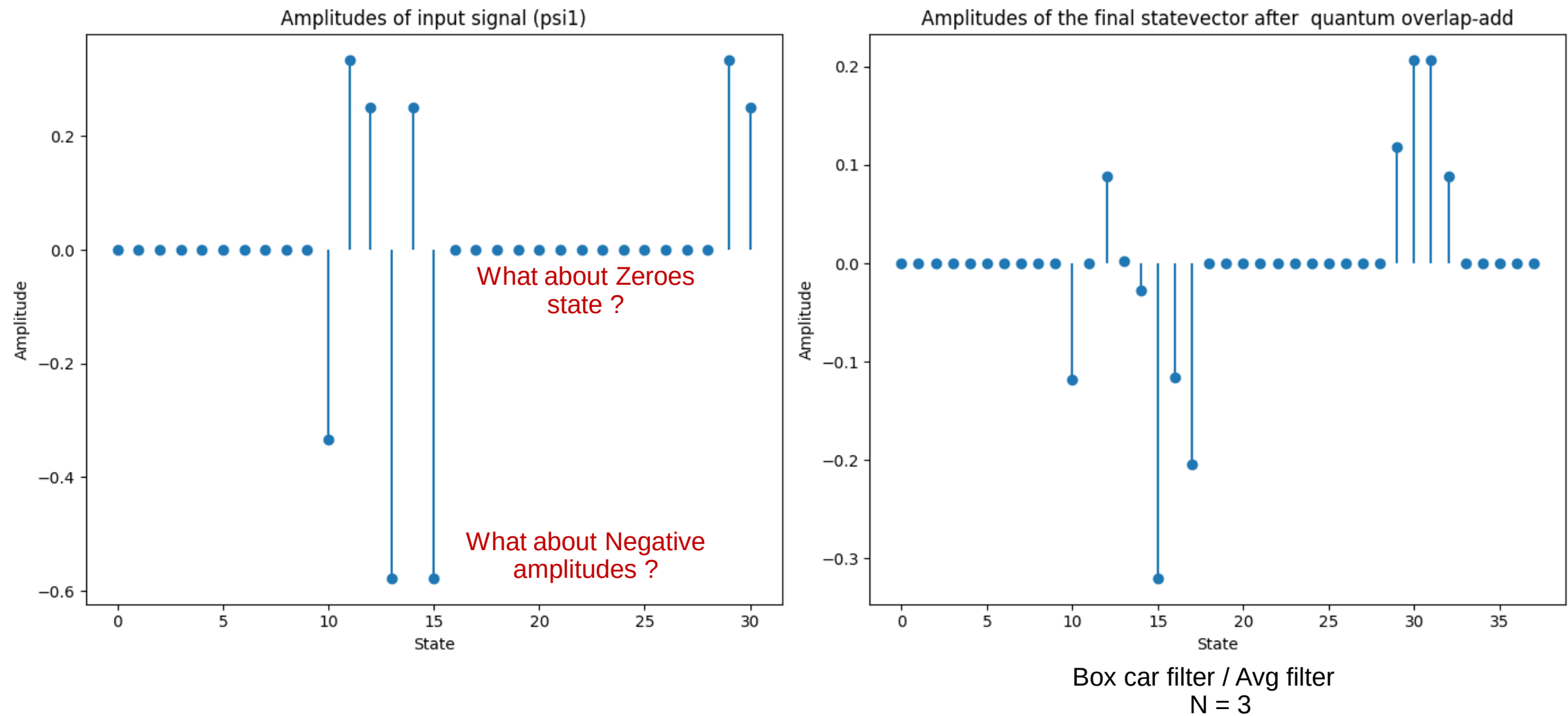
OVERLAP ADD



Classical
Post-processing

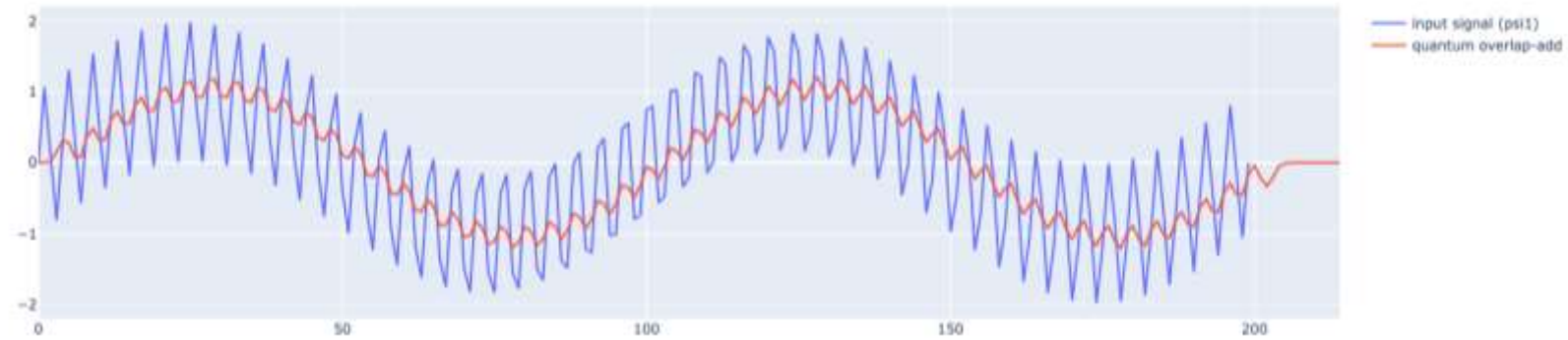


Quantum Reconstruction algorithm output:

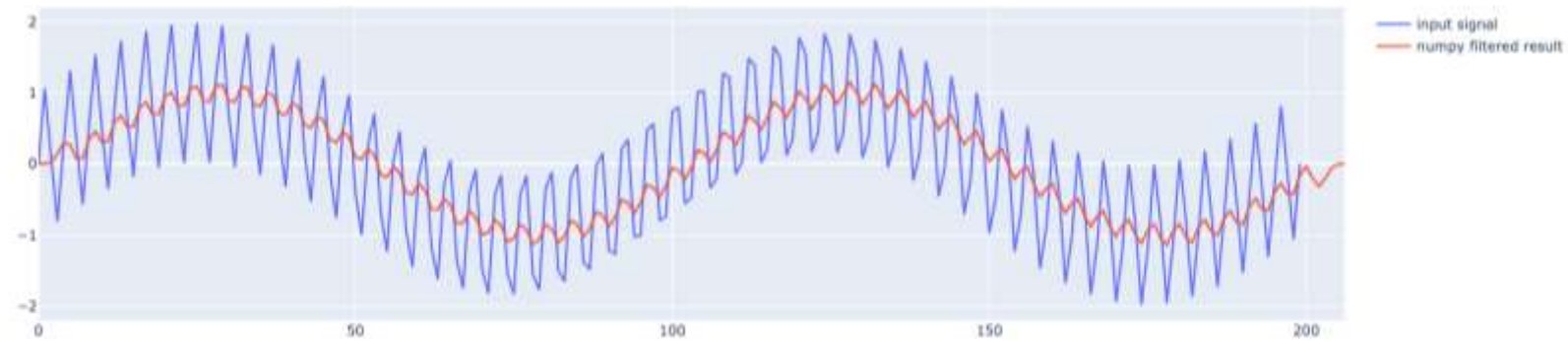


SINC FILTER (N = 8)

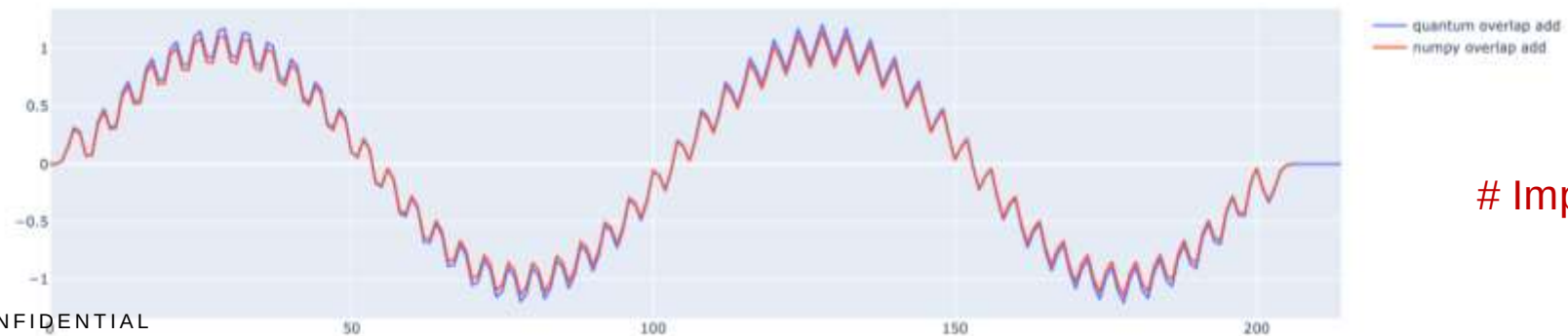
quantum overlap add with N = 8



numpy overlap add with N = 8



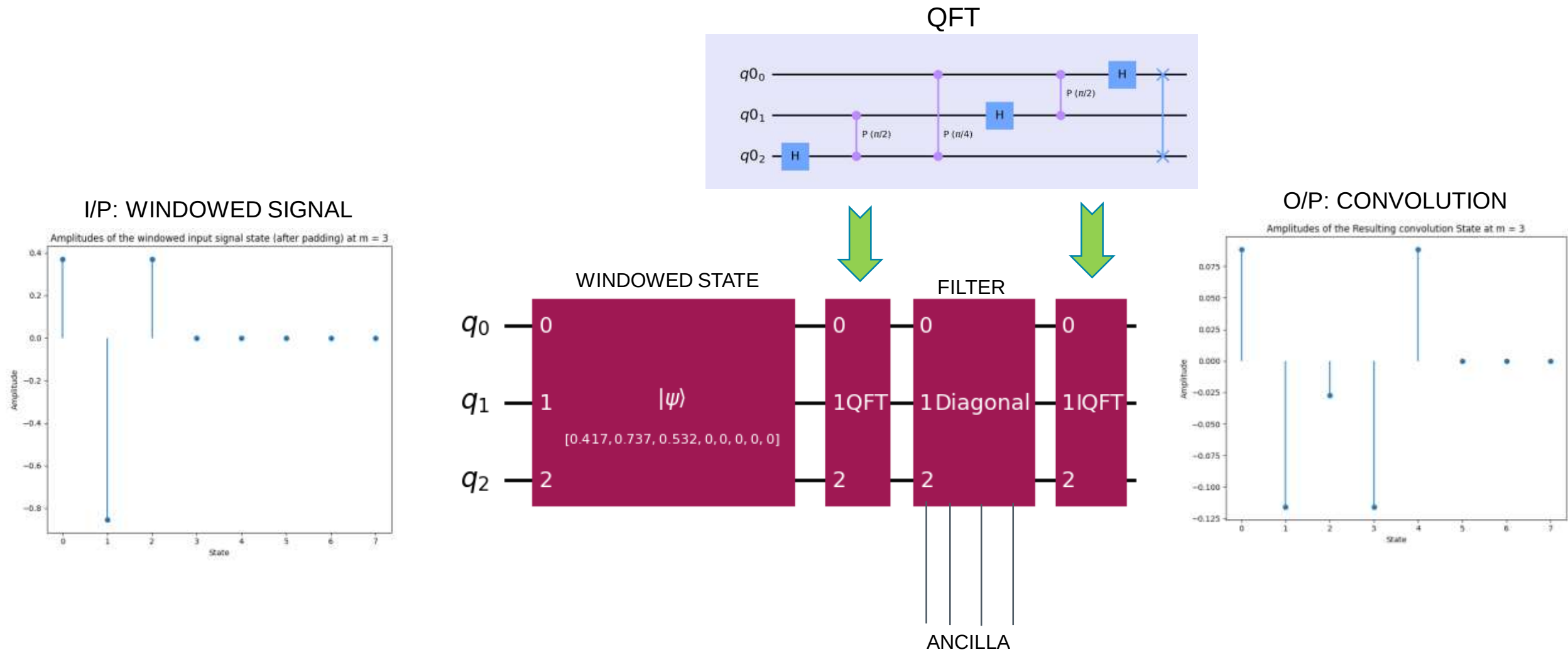
quantum vs numpy overlap add with N = 8



Implemented with Qiskit

SUBROUTINE 2: FILTER AS A UNITARY GATE

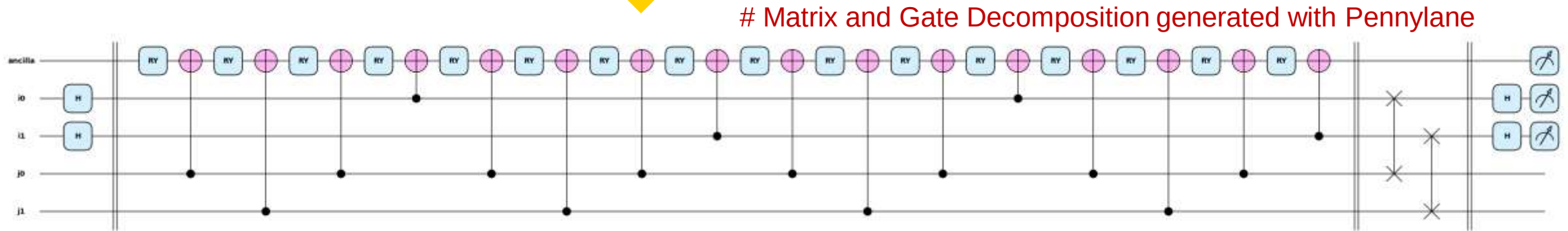
Subroutine 2: Quantum Convolution algorithm: Filter as a Unitary Gate



BLOCK ENCODING:

1. Linear Combination of Unitaries (LCU)
2. Matrix Access Oracles

$$\sum c_i U_i \rightarrow \begin{bmatrix} A & * \\ * & * \end{bmatrix}$$



1. For N qubit window register we need N+1 ancillas to handle Block encoding. Total $2N+1$ Qubits
2. Size of the Block encoded Unitary matrix would be:
 $(2^{\exp(2N+1)} \times 2^{\exp(2N+1)})$
3. Depth of circuit scales very rapidly

QFT of FILTER VECTOR

FILTER VECTOR

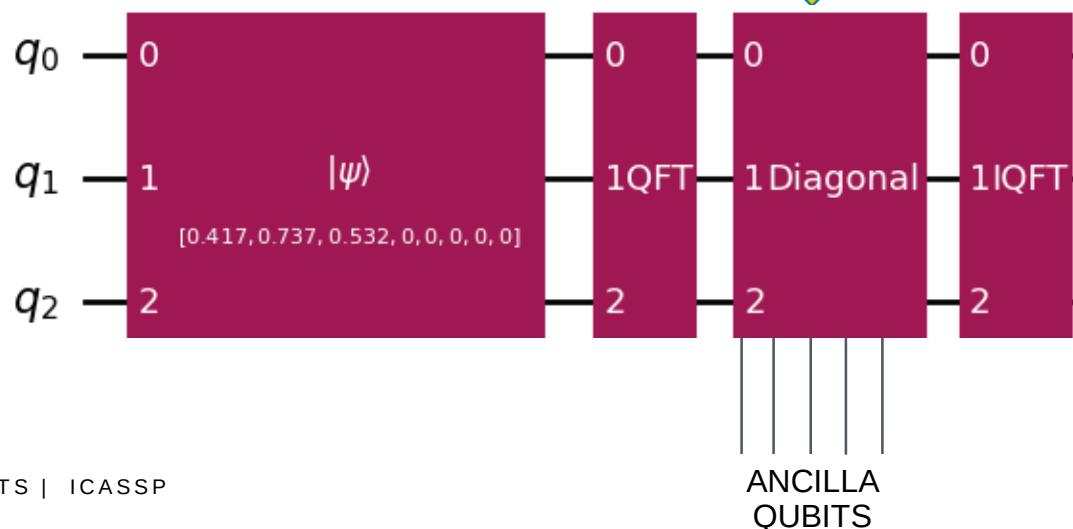
$\begin{bmatrix} 0.3333, 0.0000, 0.0000, 0.0000, \dots, -0.3776, 0.1394, 0.7503, \\ 0.0000, 0.3333, 0.0000, 0.0000, \dots, -0.3514, 0.7863, -0.3151, \\ 0.0000, 0.0000, 0.0000, 0.0000, \dots, 0.0544, -0.1298, 0.2198, \\ 0.0000, 0.0000, 0.0000, 0.0000, \dots, 0.1570, 0.1965, 0.3183, \\ \dots, \dots, \dots, \dots, \dots, 0.1901, 0.1798, 0.2232, \\ -0.2569, -0.0261, 0.0393, 0.4411, \dots, -0.6622, -0.1665, 0.1433, \\ 0.0738, 0.8639, 0.0356, 0.2536, \dots, 0.2981, -0.0821, 0.2532, \\ -0.1664, 0.3514, -0.4111, -0.4157, \dots, -0.3780, -0.4900, -0.2319 \end{bmatrix}$

2 ELEMENT FILTER VECTOR EXAMPLE

(32x32)

TODO: QSP and QSVT on filter vector encoded block unitary matrix

GATE PREPARATION



Probability factor

Quantum Convolution algorithm: Filter as a Unitary Gate Plots

16 length window/filter

Padded to 32

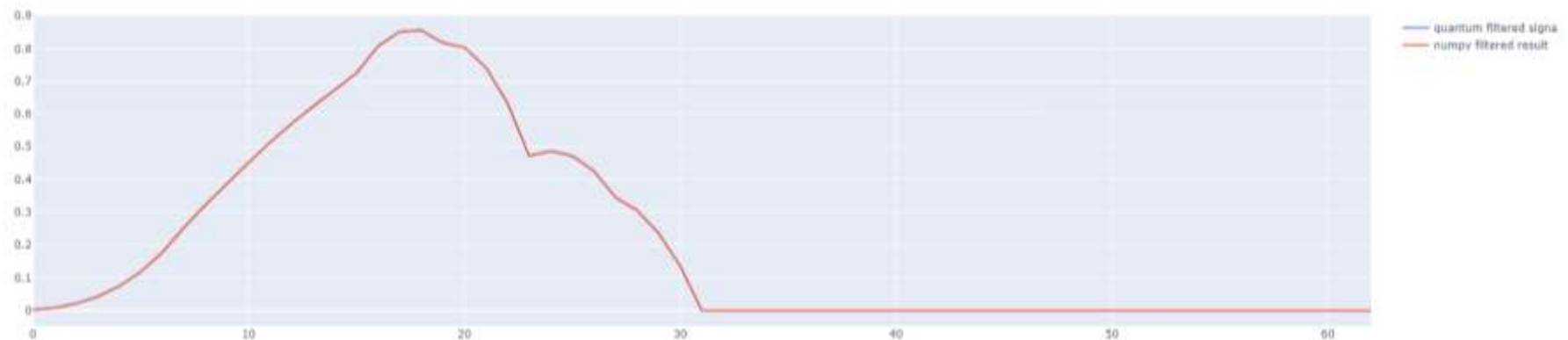
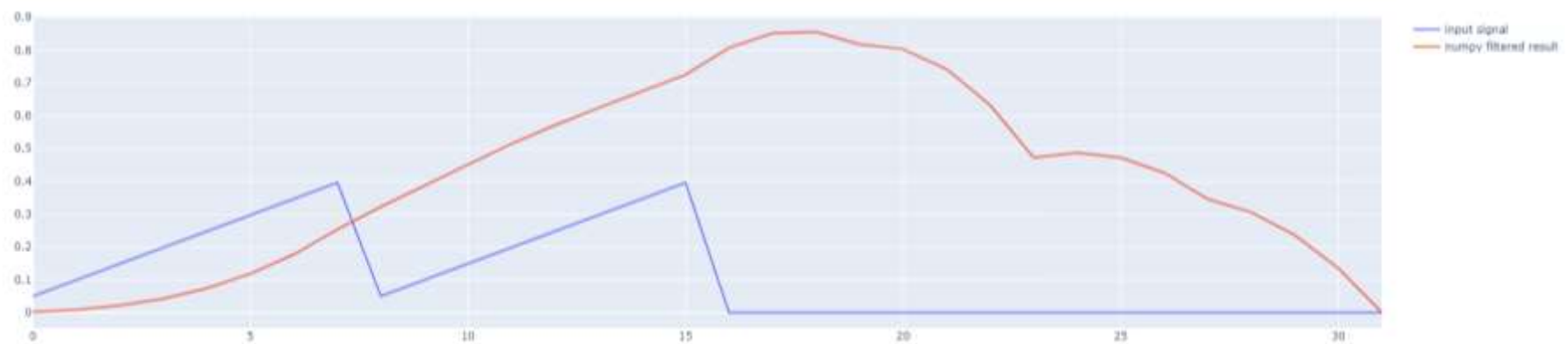
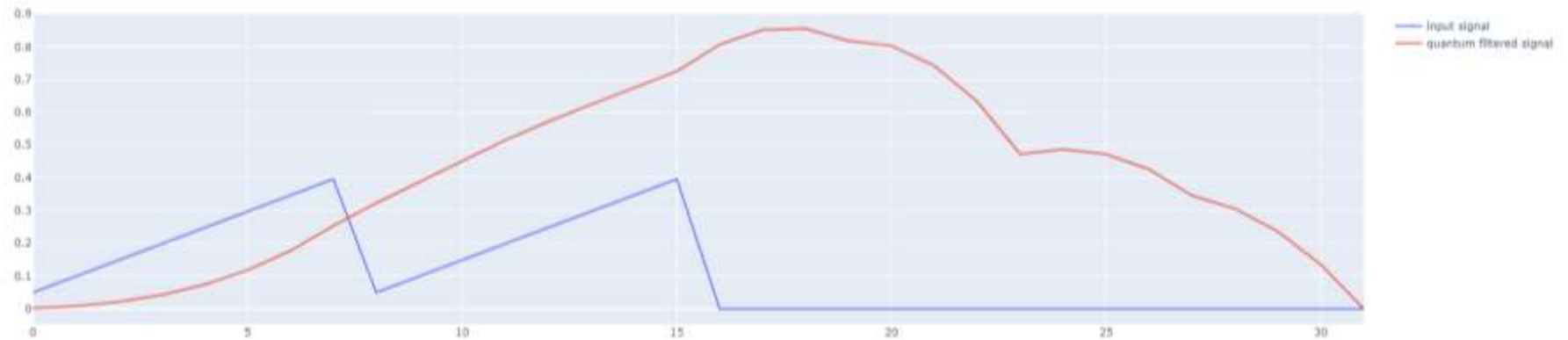
i.e. 5 Qubits

Matrix: (2048 x 2048)

ETA: 1.5 Hr

PRO: We can calculate the matrix beforehand.

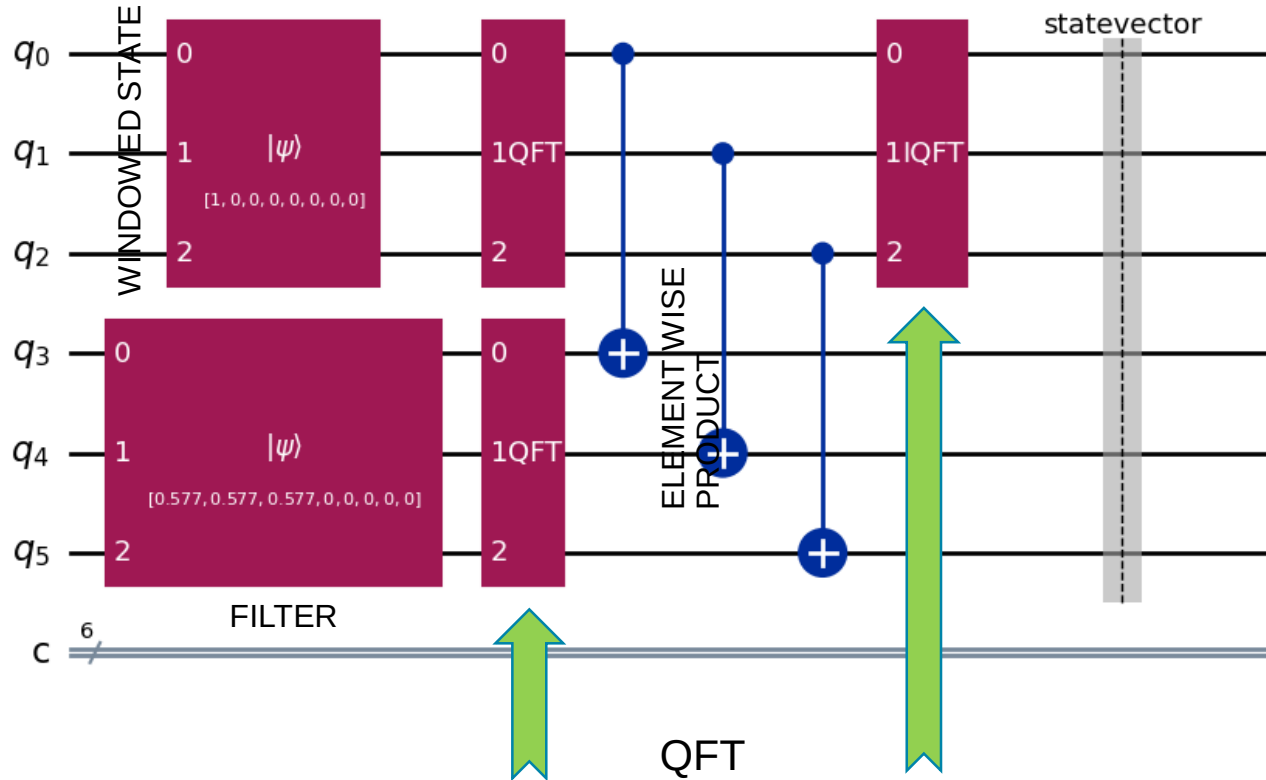
TODO: More Efficient Unitary PREP. (Sparse matrices, Structured matrices etc)



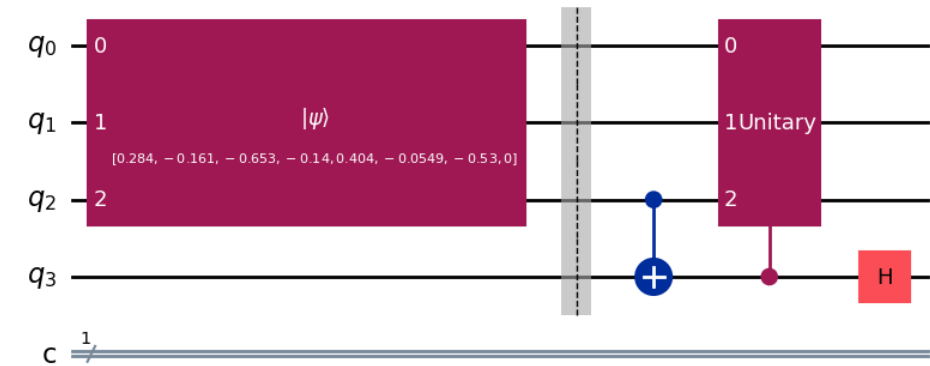
SUBROUTINE 3:
QUANTUM OVERLAP—ADD CIRCUIT

Quantum Overlap-Add

Probabilistic Algorithm

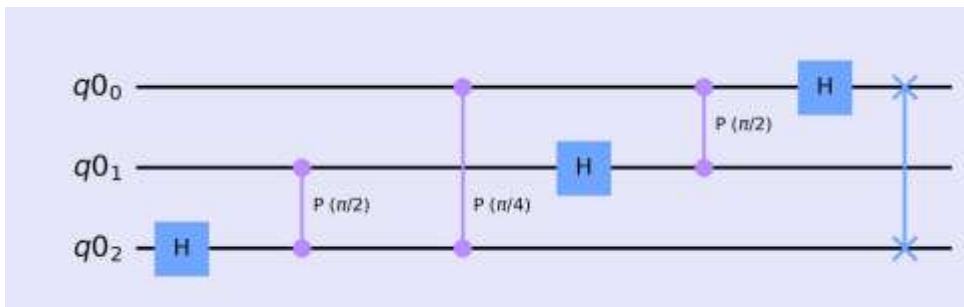


QUANTUM OVERLAP ADD

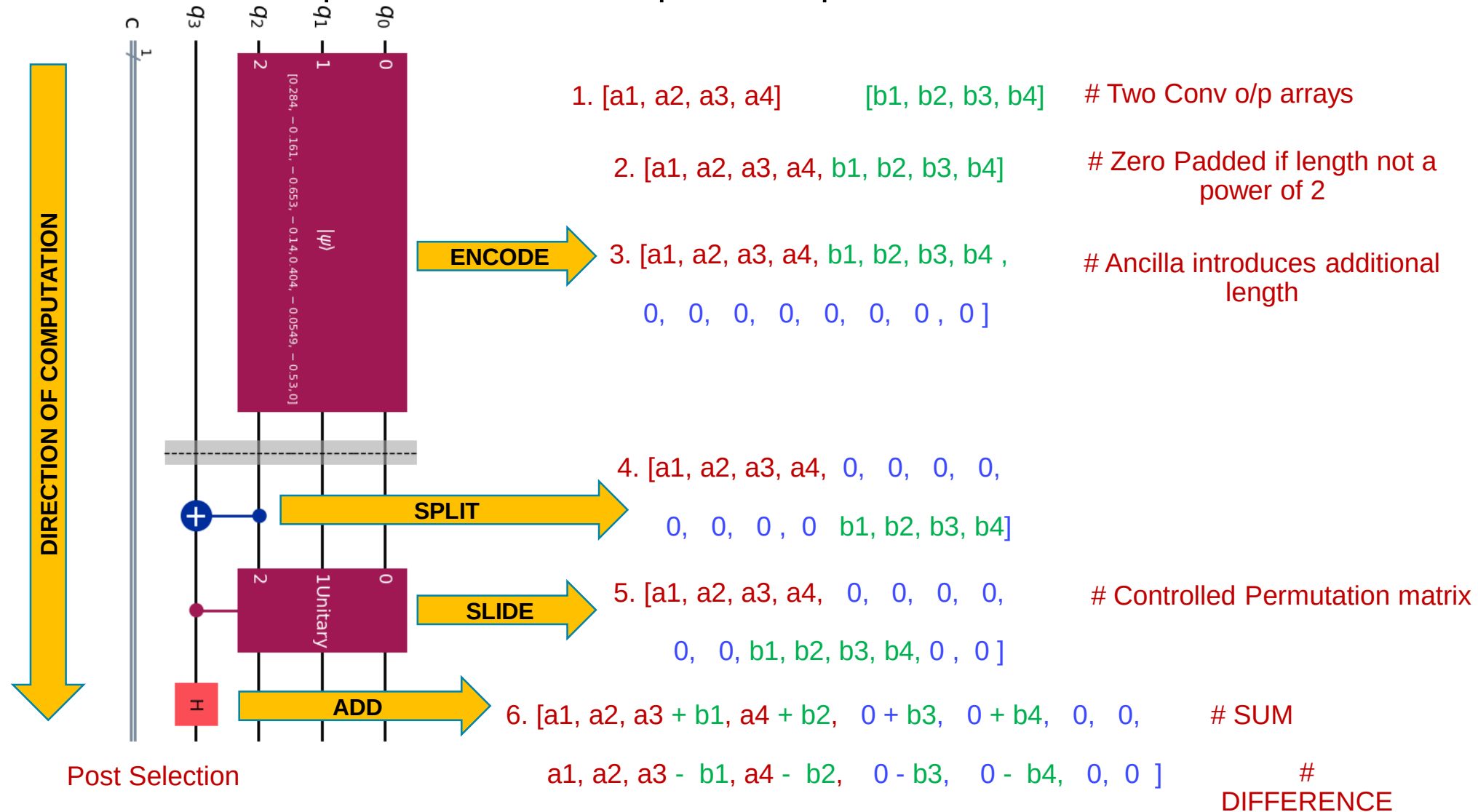


Quantum
Post-processing

For N qubit window convolution
outputs each we need total N+2
Qubits



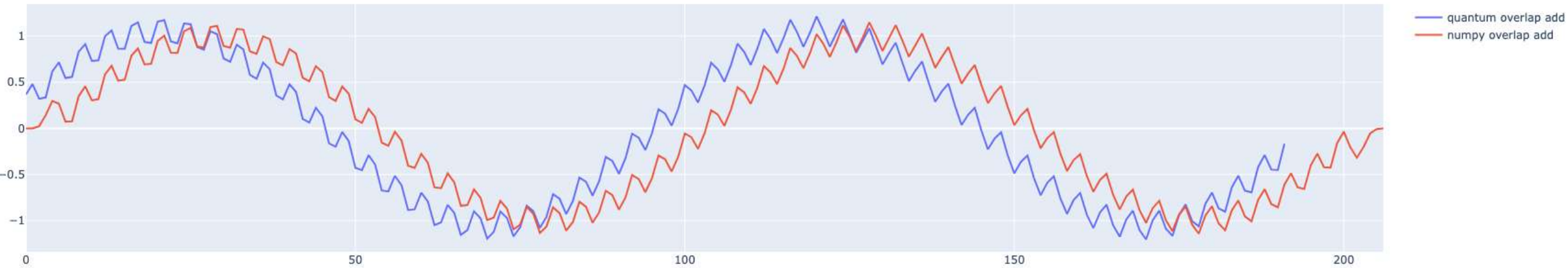
Overlap-Add Circuit Example for 4 qubits



When ancilla is measured as 0, we have our overlap add as the quantum state on remaining qubits:

Quantum Reconstruction algorithm: Overlap-Add circuit Output

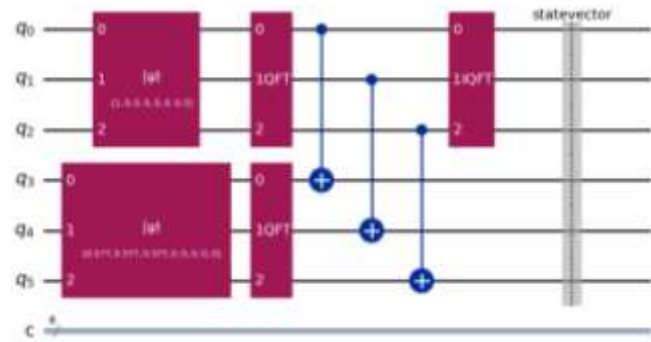
quantum vs numpy overlap add with filter lenght = 8 and window lenght = 8



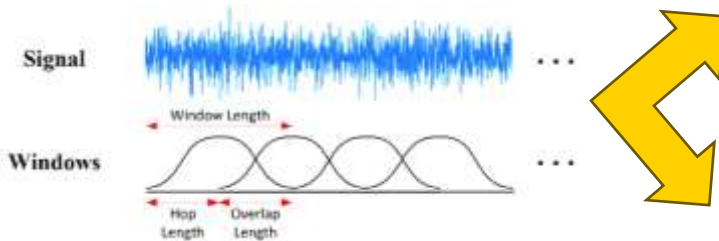
Implemented with Qiskit

FULL QUANTUM RECONSTRUCTION ALGORITHM

For N qubit window each we need total **$2(2N+1)$ Qubits**



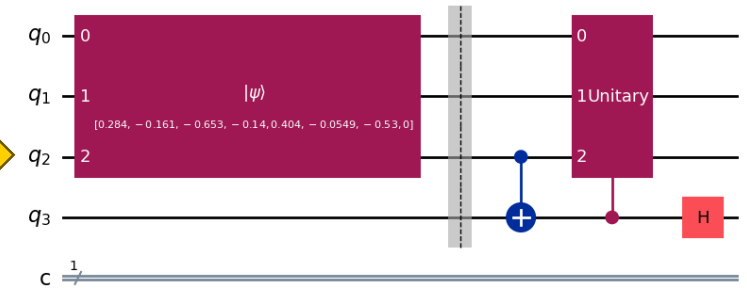
Filter as a Register



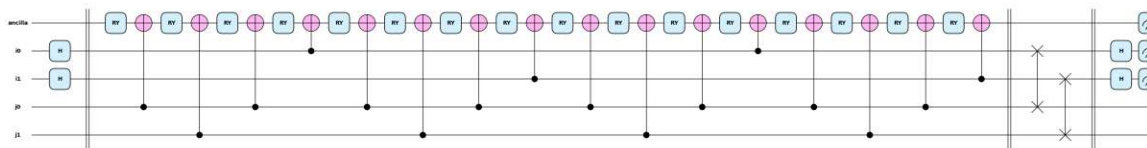
Windowing

Convolution O/P

Classical Data

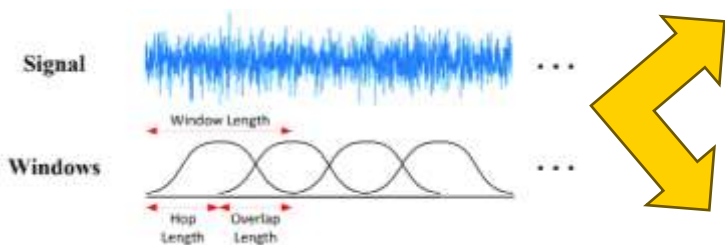


Quantum Overlap-Add

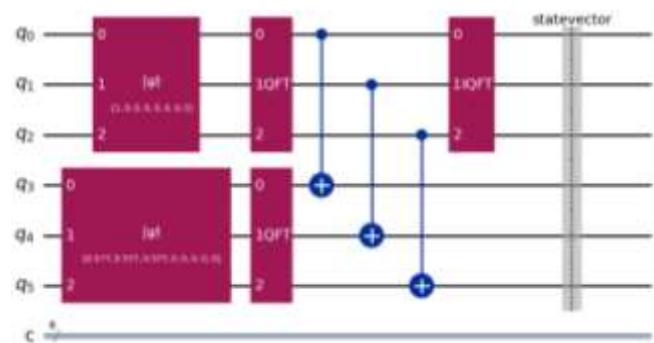


Filter as a unitary Gate

For N qubit window each we need total **$3(N+1)$ Qubits**

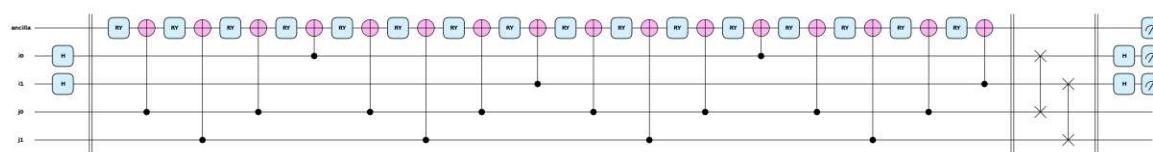


Windowing



Filter as a Register

Convolution O/P

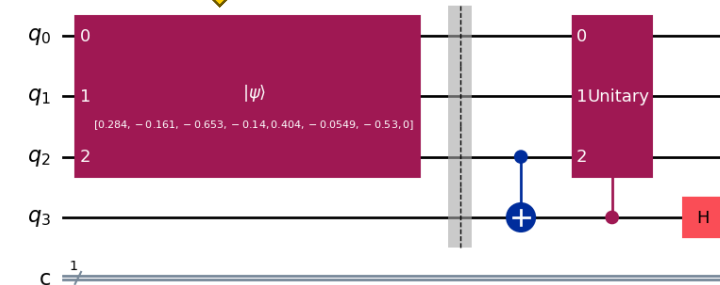


Filter as a unitary Gate

QRAM

CHALLENGES:

1. Need two copies for each window convolution O/P.
2. Different windows have different window norms.



Quantum Overlap-Add



